Basic MQL and Document Modeling

A support application must find and change the intended ticket.

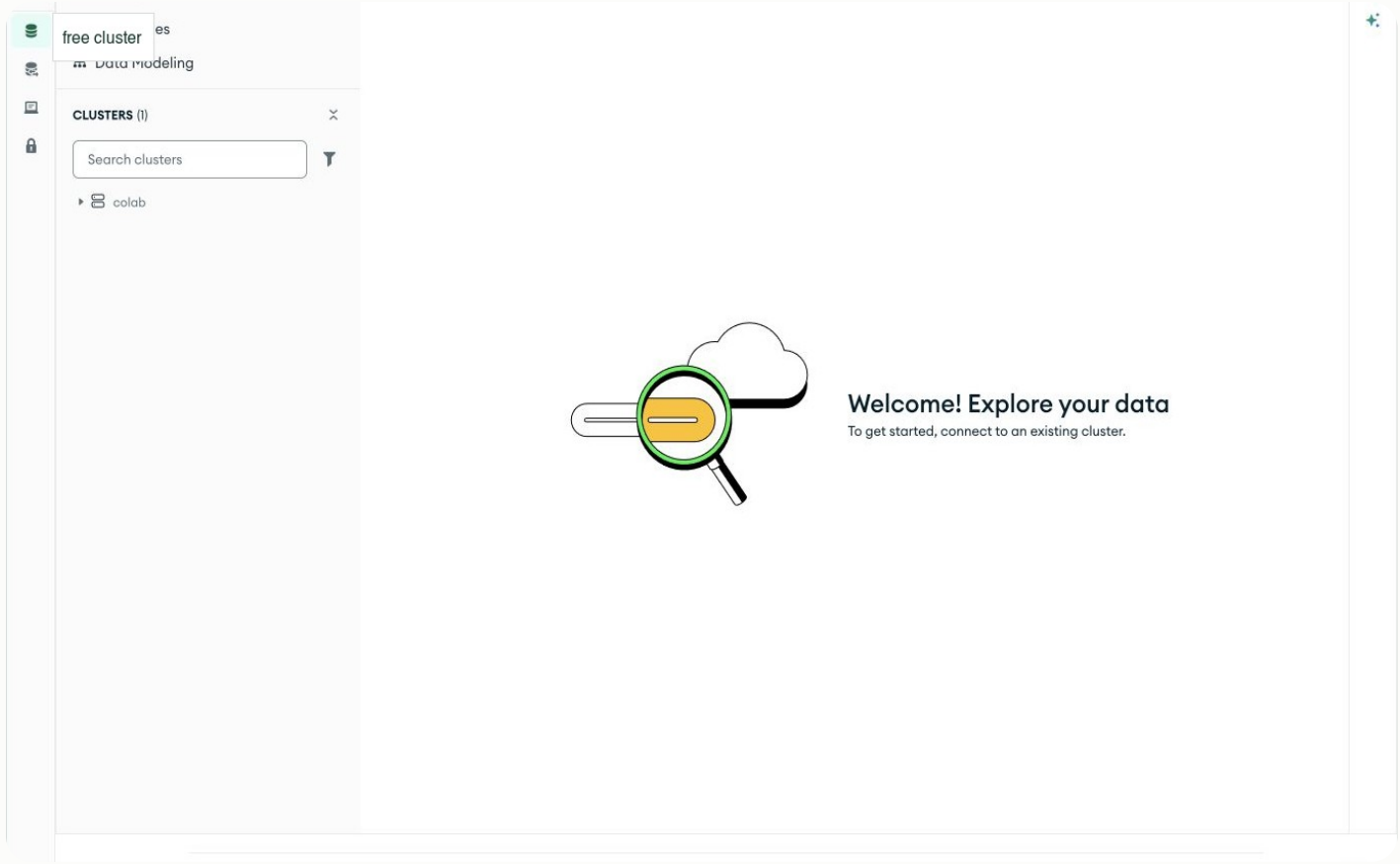
Day 1: Python queries with observable results
Day 2: document boundaries for a ticket page

SQL concepts and Python MongoDB operations

Purpose	SQL idea	Python / PyMongo
Choose a source	FROM tickets	database["tickets"]
Choose records	WHERE status = 'open'	{"status": "open"}
Choose output fields	SELECT ticket_id	{"ticket_id": 1, "_id": 0}
Order and restrict	ORDER BY ticket_id LIMIT 2	.sort("ticket_id", 1).limit(2)

The concepts transfer. Null, arrays, and BSON types need their own rules.
A filter is a dictionary, not a SQL string.

Atlas deployment and database connection



Project: cloud administration

Cluster: database service

Database: collections

Collection: documents

This capture shows a deployment while Data Explorer requests a connection.

Interface captured August 25, 2026. Names redacted. This is not a successful query result.

Colab connects from a remote runtime

Notebook action	What it establishes
Show runtime IP and practice database name	Where the code runs and where this lesson writes
Add the printed IP as a temporary /32 rule	Permit that IPv4 source to attempt a connection
Use a database user and hidden URI prompt	Authenticate separately from the Atlas website login
Send ping, then insert the teaching data	Test reachability, then the required data permissions

The notebook keeps TLS verification enabled. It never needs access from everywhere. Local mode teaches the same queries without an Atlas account.

JSON, Python, and shell notation

Value or operation	Notebook Python	mongosh
Null value	None	null
Boolean value	True / False	true / false
Find one document	tickets.find_one({...})	db.tickets.findOne({...})
Update-result property	result.modified_count	result.modifiedCount
Typed date	datetime(..., tzinfo=timezone.utc)	ISODate("2026-02-01T23:10:00Z")

The slides use Python. Chapter 10 labels its runnable shell examples.
MongoDB stores BSON, including typed dates that JSON strings do not create.

The notebook has six teaching tickets

Ticket	Category	Priority	Status
1001	streetlight	high	open
1002	sanitation	medium	in_progress
1003	water	urgent	resolved
1004	parks	low	new
1005	sanitation	high	resolved
1006	streetlight	medium	in_progress

A small adaptation of the CSV case. Event fields are simplified.
Test ticket 1099 is added later, outside these six records.

A filter and a projection

```
filter_doc = {
    "status": {"$in": ["new", "open", "in_progress"]},
    "priority": {"$in": ["high", "urgent"]}
}
projection = {"_id": 0, "ticket_id": 1, "status": 1}
for document in tickets.find(filter_doc, projection):
    print(document)
```

```
Result: {'ticket_id': 1001, 'status': 'open'}
```

Both conditions must hold. Ticket 1003 is urgent but resolved.
Projection changes the returned fields, not the stored document.

A cursor, ordering, and a limit

```
cursor = tickets.find(
    {"status": {"$in": ["new", "open", "in_progress"]}},
    {"_id": 0, "ticket_id": 1, "opened_at": 1}
).sort([("opened_at", -1), ("ticket_id", 1)]).limit(2)

for document in cursor:
    print(document)
```

Returned IDs: 1006, then 1004.
Newest opening first; ticket_id gives a tie-break order.

Iteration reads the cursor. To repeat a consumed query, call find again.

Nested fields and simple arrays

```
for document in tickets.find(
    {"requester.user_id": 101}, {"_id": 0, "ticket_id": 1}
):
    print(document)

for document in tickets.find(
    {"tags": "safety"}, {"_id": 0, "ticket_id": 1}
):
    print(document)
```

Both queries return IDs 1001 and 1005 for different reasons.
The dotted name reaches a nested field. The tag condition matches an array value.

Two conditions can match different events

Ticket 1001 event	type	actor_role
5001	created	resident
5002	assigned	agent

```
wrong_question = {
    "events.type": "created",
    "events.actor_role": "agent"
}
```

Ticket 1001 matches this filter, but no single event says an agent created it.

Static diagram on slide 30

$elemMatch requires one qualifying event

```
agent_created = {
    "events": {
        "$elemMatch": {"type": "created", "actor_role": "agent"}
    }
}
print(list(tickets.find(agent_created, {"_id": 0, "ticket_id": 1})))
```

Result: []
The empty result correctly answers the question for this fixture.

Changing type to status_changed returns tickets 1002 and 1003.
A matching document can still contain other, nonmatching events.

Static diagram on slide 30

One disposable ticket for write practice

```
test_filter = {
    "ticket_id": 1099, "test_record": True, "course_fixture": "cst4714"
}
tickets.delete_many(test_filter)
tickets.insert_one({
    **test_filter, "status": "new", "subject": "Disposable MQL test",
    "events": []
})
```

This reset only targets marked test ticket 1099.
The notebook also creates a unique index on ticket_id.

Run after the connection and six-ticket setup. No sample database is changed.

A repeated $set can match without changing

```
result = tickets.update_one(
    test_filter,
    {"$set": {"status": "in_progress", "assignee_id": 202}}
)
print(result.matched_count, result.modified_count)
print(tickets.find_one(test_filter, {"_id": 0, "status": 1}))
```

Execution	Matched	Modified	Stored status
First	1	1	in_progress
Same update again	1	0	in_progress

Static diagram on slide 31

An expected-state condition can prevent a write

```
result = tickets.update_one(
    {**test_filter, "status": "new"},
    {"$set": {"status": "resolved"}}
)
print(result.matched_count, result.modified_count)
print(tickets.find_one(test_filter, {"_id": 0, "status": 1}))
```

Result: 0 0. Status remains in_progress.
The current document no longer satisfies status = new.

A zero match is an application outcome to interpret, not proof of a connection failure.

Static diagram on slide 32

Appending an event and repeating the request

```
result = tickets.update_one(
    {**test_filter, "events.event_id": {"$ne": 5999}},
    {"$push": {"events": {
        "event_id": 5999, "type": "status_changed",
        "actor_role": "agent"
    }}}
)
print(result.matched_count, result.modified_count)
```

First request: 1 1. Repeated request: 0 0.
The filter refuses a second append of event 5999.

This append and the earlier status update were separate writes.
Day 2 combines related changes in a single update.

Deletion with an exact target

```
print(tickets.find_one(test_filter, {"_id": 0, "ticket_id": 1}))
result = tickets.delete_one(test_filter)
print("Deleted:", result.deleted_count)
print("Remaining test record:", tickets.find_one(test_filter))
print("Other tickets:", tickets.count_documents(
    {"course_fixture": "cst4714"}
))
```

Deleted: 1. Remaining test record: None. Other tickets: 6.
Repeating the same delete reports 0.

An empty filter matches everything. Preview and deletion are separate operations, so a preview alone is not a lock against concurrent changes.

Lab 1: the lighting and sanitation queue

Show active streetlight or sanitation tickets, newest first.
Include priority so the crews can plan their work.
Explain why high-priority ticket 1005 stays out of this queue.

Then compare the array queries and repeat the $set update.
Read the changed test ticket before deleting it.

Individual work. Submit the completed notebook with one short explanation.
Run cleanup before finishing. No extra report or Atlas screenshot.

Day 2: the ticket-page workload

The page needs current status
and only its latest events.

Contact details change independently.
The system must keep years of event history.
Those requirements influence the document boundary.

Relationship cardinality and stored shape

Relationship in this application	Cardinality	A modeling consequence
Ticket to requester	Many tickets to one user	A shared user can change independently
Ticket to events	One ticket to many events	History can outgrow an embedded array
Publisher to headquarters	One to one, by assumption	A small owned address can fit in its publisher

Cardinality describes how entities relate. It does not choose the storage by itself. Ownership, reads, writes, and growth still matter.

One-to-one modeling: the live example

MongoDB University
Modeling Data Relationships
Lesson 3: Modeling One-to-One

```
{
  "publisher_id": "p7",
  "name": "Example Press",
  "headquarters": {"city": "New York", "country": "US"}
}
```

This example publisher owns one small headquarters address.
Student practice uses Lesson 4: Modeling One-to-Many, a different activity.

Status and history in one document update

```
result = demo_tickets.update_one(
    {"ticket_id": 1001, "status": "open"},
    {
        "$set": {"status": "in_progress"},
        "$push": {"events": {
            "event_id": 5099, "type": "status_changed"
        }}
    }
)
print(result.matched_count, result.modified_count)
```

The status change and event append form one atomic document write. Repeating it finds no open ticket 1001: 0 0, without a second event.

Instructor demo uses a fresh collection. Separate documents need separate coordination.

Current contact details and historical snapshots

Stored fact	Owner	Effect of a contact correction
requester_id: 101	Ticket stores the relationship	ID stays the same
Current email for user 101	Authoritative user document	Update one user record
requester_name_at_open	Ticket owns a historical snapshot	Keep it only if the historical meaning is intended

The page can read the user by ID for current contact details.
A copied current email would introduce another value to maintain.

A reference on each event allows independent growth

```
{"ticket_id": 1001, "requester_id": 101, "status": "open"}
```

```
{
  "event_id": 5002,
  "ticket_id": 1001,
  "event_type": "assigned",
  "event_at": "2026-02-02T14:05:00Z"
}
```

Each event points to its ticket. The parent does not need an ever-growing ID list.
The timestamp above is JSON text; the runnable demo stores a BSON date.

Reading the ticket page from referenced data

```
ticket = page_tickets.find_one({"ticket_id": 1001}, {"_id": 0})
person = people.find_one({"user_id": ticket["requester_id"]}, {"_id": 0})
recent = list(history.find(
    {"ticket_id": ticket["ticket_id"]}, {"_id": 0}
).sort([("event_at", -1), ("event_id", -1)]).limit(2))
print(ticket["status"], person["display_name"])
print([event["event_id"] for event in recent])
```

Worked demo: open, Requester 101, then event IDs [5002, 5001].
Three reads assemble one page. They are not one atomic snapshot.

An event index starting with ticket_id and then the ordering fields can support this read.

Growing history and a bounded preview

Representation	What grows with each new event	Page-read consequence
All events embedded	The ticket document	One read, increasing document size
All events separate	The number of event documents	Ticket plus bounded history query
Separate history and a recent copy	History, but recent copy stays capped	Convenient preview with refresh work

A BSON document has a 16 MiB size limit.
A design can become awkward well before that limit.

A capped copy needs an explicit refresh and stale-data policy.

A model decision has an observable consequence

A separate user record lets a contact correction happen once.
Separate event records keep complete history without enlarging the ticket.

The page must retrieve the related records. If it needs one consistent version of all of them, the application needs stronger coordination.

Workloads matter in PostgreSQL too. PostgreSQL supports arrays and JSON.
Neither a SQL table nor a MongoDB collection chooses a good model for us.

Lab 2: the latest two events and all history

MongoDB University: Modeling Data Relationships
Lesson 4: Modeling One-to-Many > Practice
The Week 10 guide links directly to the lesson and activity.

Revise one JSON design for ticket 1003.
The page shows the latest two events, newest first.
Retain all history and keep current contact details editable in one place.

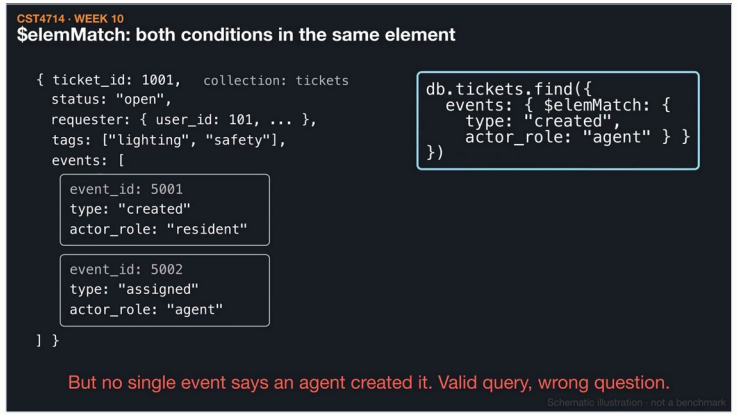
Individual Brightspace text response: JSON, a short developer explanation,
and the Lesson 4 Practice progress image. No complete course badge.

Explaining the skill in professional language

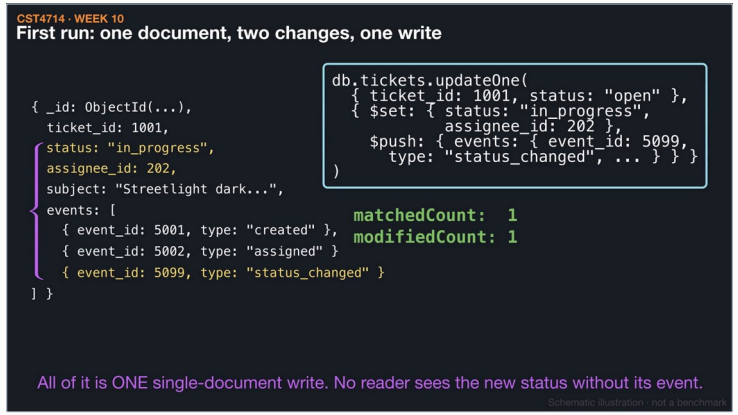
I tested a document query against a small fixture and used a counterexample to catch an array filter whose conditions matched different events. I checked matched and modified counts, then read the document back. For a ticket page, I compared where shared contact details and growing history should live, including the extra reads and consistency tradeoffs.

Describe the experiment you actually performed.
Next week: aggregate results and enforce document rules.

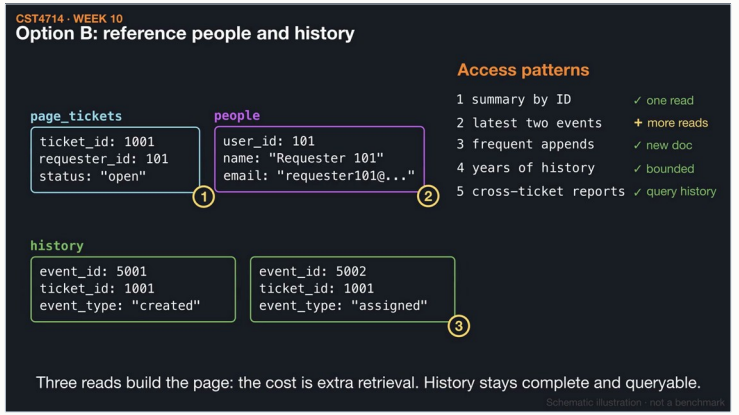
This week's videos



[Querying inside documents: arrays and $elemMatch (1:12)]



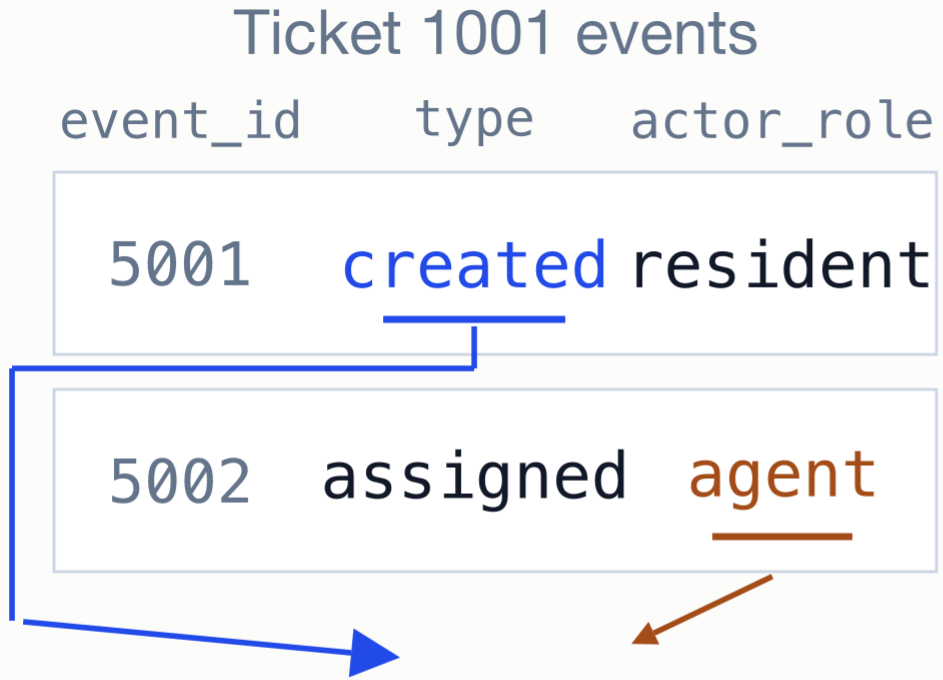
[Did the write do what you meant? (1:06)]



[Embed or reference? Let the workload decide (1:21)]

In PowerPoint Slide Show, click a thumbnail to play full screen. Embedded videos work offline.

Array conditions and one matching event



Separate dotted conditions

Both values occur somewhere

Ticket matches



$elemMatch

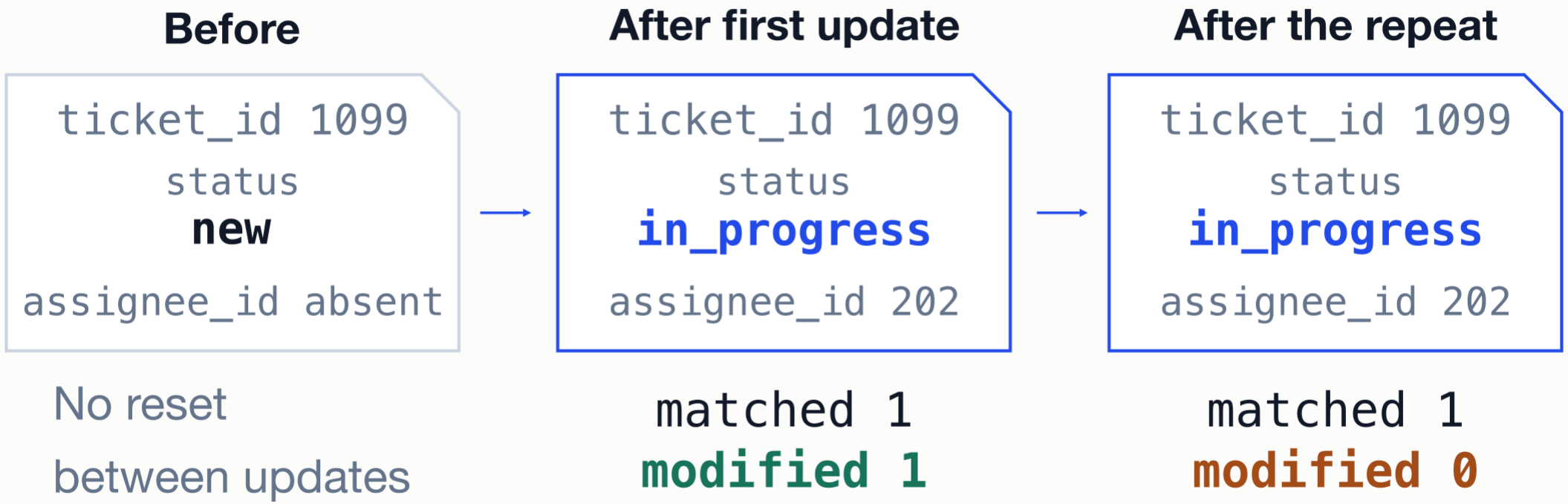
No event contains both values

Ticket does not match

Return to slide 11

The first update and the repeat

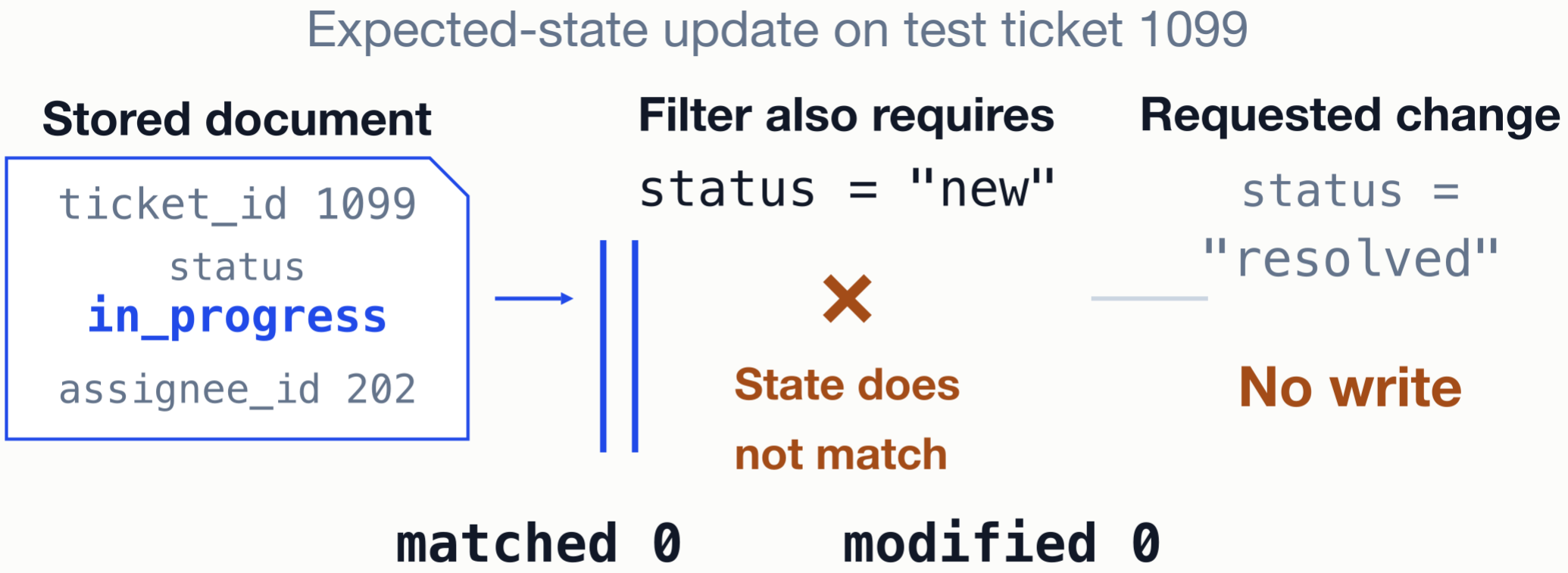
```
$set  status = "in_progress", assignee_id = 202
```



The second filter still matches. The requested values are already stored.

Return to slide 13

The expected state in an update filter



Status stays in_progress. Reload the document before deciding what to do next.

Return to slide 14